CAPSTONE · SYSTEM OVERVIEW

MetMemo: how it works and how it is built

A digital business card with an AI assistant that remembers everyone you meet and tells you when to follow up.

Met

The people you meet: your card, its QR code, the lead form, and paper cards you scan.

Memo

The memory of each meeting: your notes, follow-up dates, and the AI that recalls them and drafts your follow-up.

01 The workflow, step by step

One full loop, from making your card to sending a follow-up. The AI only suggests and drafts. You decide and you send.

01 **Sign up and create your card**

Name, job title, company, email, phone, website, location, bio, and your profile photo. You can make more than one card.

02 **Get your link and QR code**

Each card gets a public page at `/c/your-name` and a QR code that opens it.

03 **Meet someone**

At an expo, a client visit, or a job fair. There are three ways the person becomes a contact.

04 **a. They scan your QR**

They see your card, can save you to their phone (vCard), and can leave their name, number and a message on the lead form.

05 **b. They hand you a paper card**

You take a photo. The AI card scanner reads the fields and flags any it is unsure about for you to confirm.

06 **c. You add them by hand**

For people who had no card at all.

07 **The contact is saved**

With where they came from (lead form, scan, or manual), the card that captured them, and a follow-up date two days out.

08 **Write a note after the meeting**

What you talked about, what they need, what you promised. These notes are the memory the AI searches later.

09 **Check your dashboard**

Shows your cards, your contacts, and who is due for follow-up.

10 **Ask the assistant**

"Who should I follow up with this week?" or "What did I talk about with Juan?" The agent plans, calls tools, and searches your notes.

11 **Review the drafts and send them yourself**

Edit each draft message and send it from your own email or phone. Then mark the contact as contacted and set the next follow-up date.

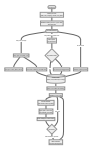


Figure 1. User workflow. Rounded box is the start, diamonds are decisions, the cylinder is the saved contact.

02 System architecture

Two applications. The Laravel app owns the users, the data, and every page. The Python AI service owns every LLM call. Laravel never calls OpenAI directly; it calls the AI service over HTTP with a shared secret header.

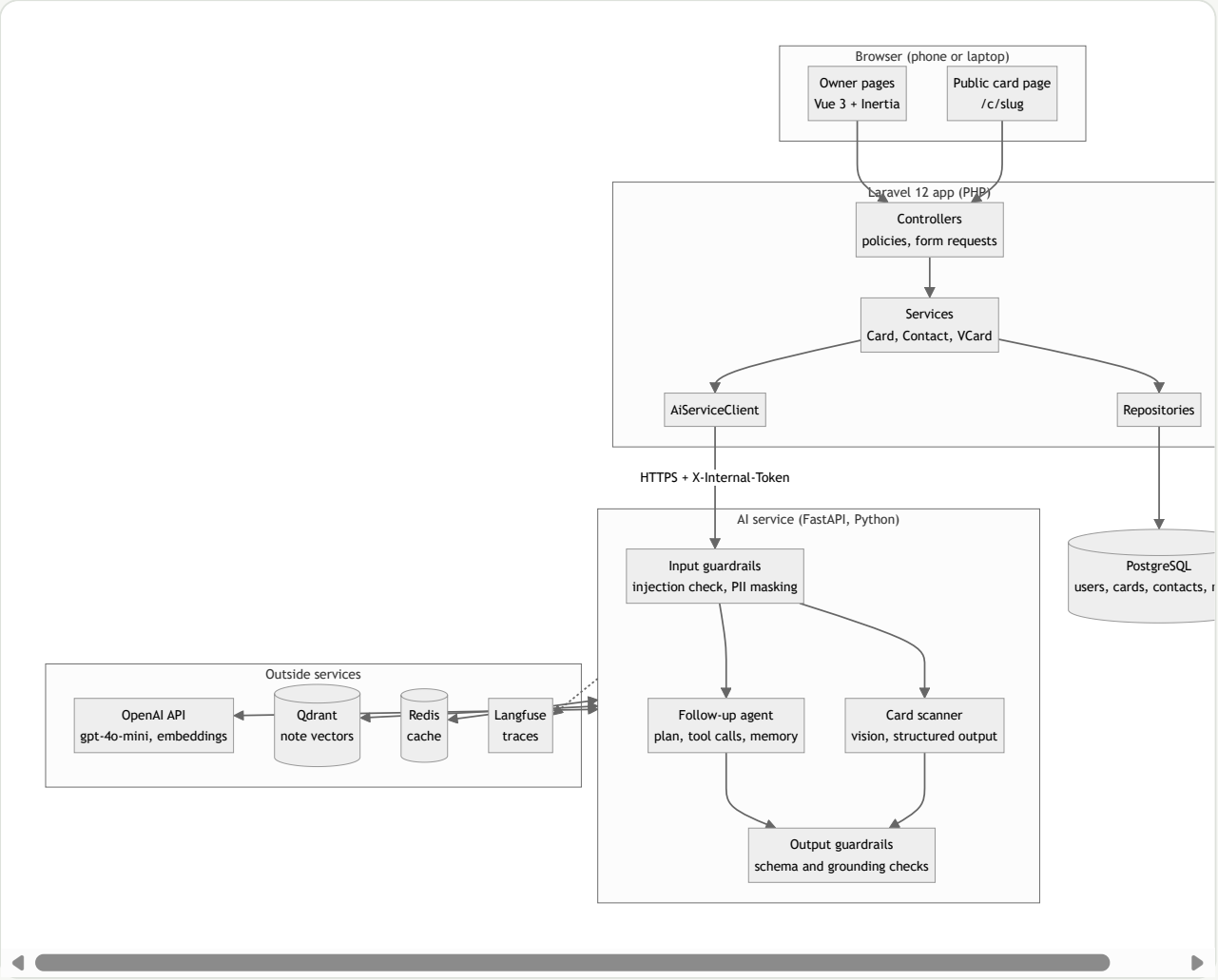


Figure 2. Components and data flow, top to bottom. The dotted line is tracing. The agent's contact tools read data through a read-only Laravel endpoint scoped to the signed-in user, so ownership rules stay in one place.

PART	TECHNOLOGY	JOB
Web app	Laravel 12, Jetstream, Inertia, Vue 3, Tailwind	Login and 2FA, cards, contacts, notes, dashboard, public card, QR and vCard
Database	PostgreSQL	All app data; every contact and card belongs to one user
AI service	FastAPI, Pydantic	The only place that talks to the LLM; validates every request and response
LLM	OpenAI gpt-4o-mini	Plans, chooses tools, reads card photos, drafts messages
Embeddings	text-embedding-3-small	Turns notes into vectors for search
Vector store	Qdrant	Finds the notes that match a question (RAG), filtered to one user
Cache	Redis	Reuses answers and embeddings for repeated questions
Tracing	Langfuse	Shows each request's tool calls, retrieved notes, latency, and output
Deployment	Docker (AI service has a Dockerfile)	Runs both apps on a server at a public URL

03 Code architecture inside Laravel

Every request goes down the same four layers. Each layer has one job, which keeps the code testable and easy to explain in a design review.

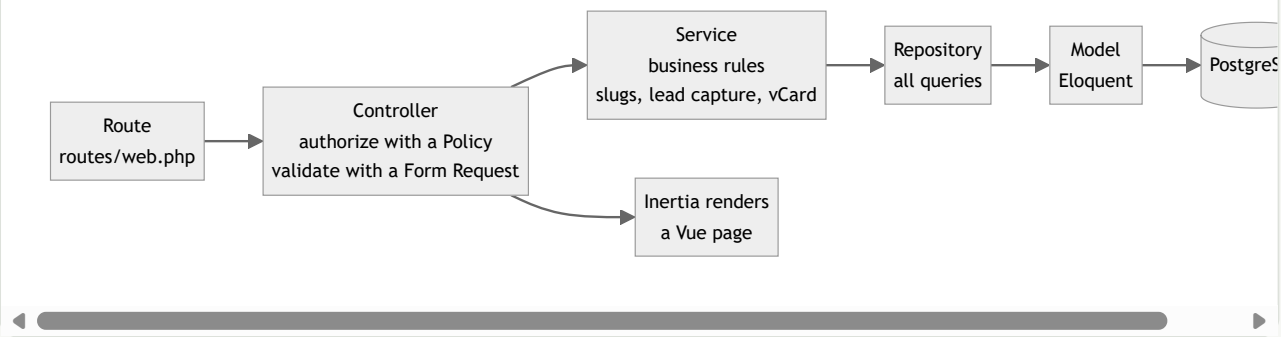


Figure 3. Request path through the Laravel app.

LAYER	FOLDER	EXAMPLES
Controller	`app/Http/Controllers`	CardController, ContactController, PublicCardController, DashboardController
Policy	`app/Policies`	CardPolicy, ContactPolicy: you can only touch your own records
Service	`app/Services`	CardService (unique slugs), ContactService (lead capture, due list), VCardService, AiServiceClient
Repository	`app/Repositories`	Interfaces in Contracts, Eloquent versions bound in RepositoryServiceProvider
Model	`app/Models`	User, Card, Contact, ContactNote
Pages	`resources/js/Pages`	Dashboard, Cards, Contacts, Public/Card, Profile, Welcome

04 How one AI request works

This is the agentic part. The assistant does not answer in one shot. It plans, calls tools to get real data, searches your notes, and only then drafts. Guardrails check what goes in and what comes out.

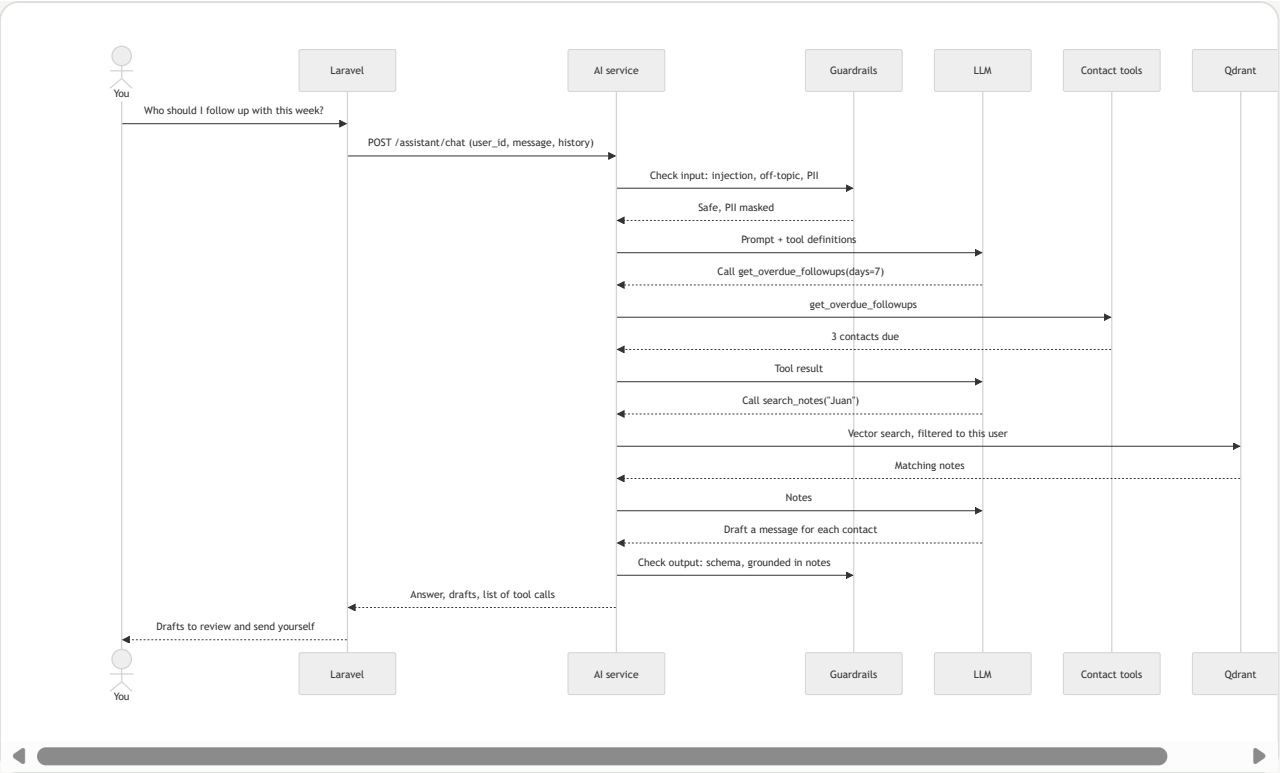


Figure 4. Follow-up assistant request, including tool calls and retrieval.

SPEC ITEM	HOW METMEMO COVERS IT
Tool calling	`get_overdue_followups`, `get_contact_notes`, `search_notes`, `draft_followup_message`
Multi-step planning	Find who is due, read their notes, then draft, in one request
RAG	Search over your own contact notes in Qdrant, always filtered to your user id
Memory	The chat history is sent with each message
Structured output	The card scanner returns each field with a confidence score and a needs-review flag
Human in the loop	Scanned fields are confirmed by you; drafts are never sent automatically
Graceful failure	If the AI service is down, the dashboard shows it as offline and the rest of the app keeps working

Guardrails

Caching

Contacts are full of phone numbers and emails, so PII is masked before it reaches the LLM or the logs. A visitor's lead-form message becomes a note the assistant later reads, so it is checked for prompt injection such as "ignore your instructions".

Redis stores answers to repeated questions and the embeddings of notes that have not changed. Editing a note clears its cached embedding.

Evaluation

At least 15 scripted assistant scenarios, including failure cases, for tool-selection accuracy and task completion. RAG metrics on note questions. Field-level accuracy for the scanner on real card photos.

05 Database

Four core tables. Every card and every contact belongs to exactly one user, which is how privacy is enforced. A contact may point to the card that captured it.

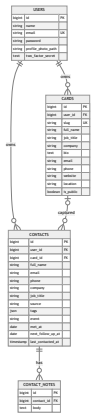


Figure 5. Entity relationship diagram. All tables also have created_at and updated_at.

TABLE	WHAT IT HOLDS	NOTES
users	Accounts	Jetstream login, two-factor auth, profile photo
cards	Your digital business cards	`slug` is the public link; `is_public` turns the link off
contacts	Everyone you met	`source` is manual, lead_form, or scan; index on (user_id, next_follow_up_at) for the due list
contact_notes	What you talked about	The text the AI embeds and searches
Qdrant: notes	One vector per note	Payload: user_id, contact_id, note_id, so every search is filtered to one user
Framework tables	sessions, cache, jobs, personal_access_tokens, passkeys	Created by Laravel and Jetstream

06 Build status

What runs today and what is still to build, so the demo and the proposal match the code.

FEATURE	STATUS	DETAIL
Accounts, profile photo, 2FA	Built	Jetstream
Cards, public page, QR, vCard	Built	Profile-only cards; private cards return 404
Lead form to contact	Built	Saves message as a note, follow-up in 2 days, rate-limited
Contacts, notes, dashboard	Built	Due-for-follow-up list
AI service skeleton	Built	Health check, internal token, request and response schemas
Card scanner	Built	Photo to reviewed contact; gpt-4o-mini vision with strict JSON output; needs an OpenAI key to run
Contacts by card	Built	Filter contacts by the card that captured them
Follow-up assistant	To build	Tools, RAG over notes, memory
Guardrails, caching, tracing	To build	Config for Redis, Qdrant and Langfuse is in place
Evaluation and deployment	To build	Scenario suite, RAG metrics, public URL